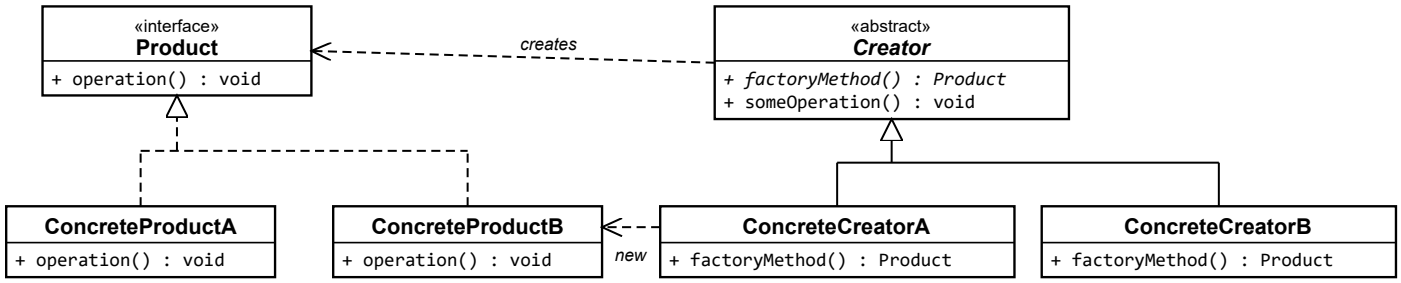
02 Factory Method

Creational

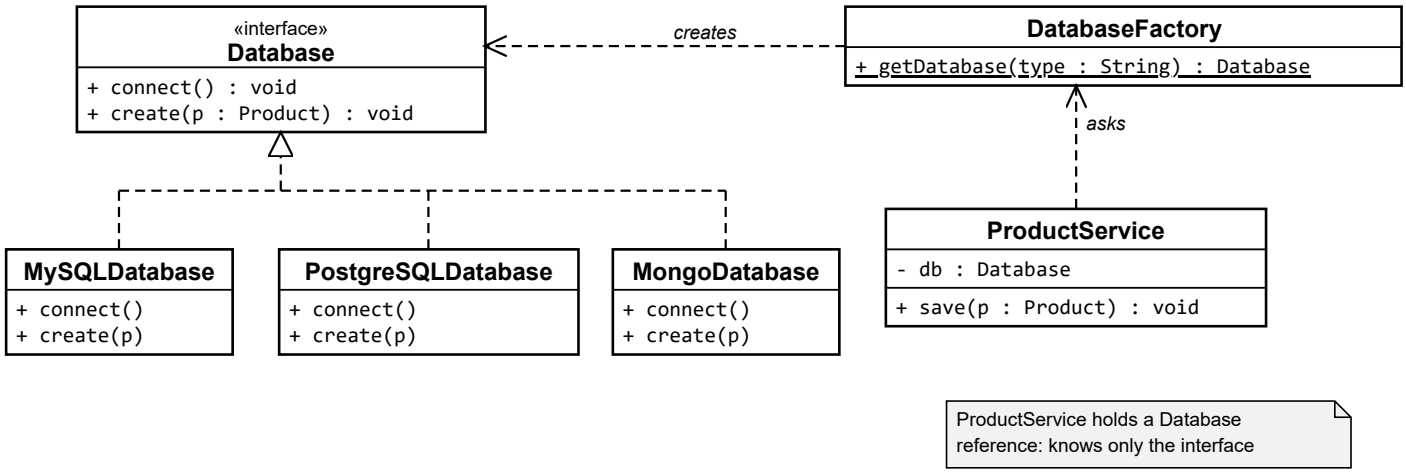
Intent (GoF)	Define an **interface for creating an object**, but let **subclasses decide which class to instantiate**. Factory Method lets a class defer instantiation to subclasses.
Use when	Several related types share one interface and **which one to create depends on input** (type, user choice, config). Client should not know concrete classes / should not use `new`.
Trigger words	“create different types of” (payment objects, book objects, accounts, DBs, LMS modules), “depending on user request / need”, “without exposing the class”, “too many if-else for object creation”, “scalable, extensible product-based framework”.
Participants	**Product** (interface) · **ConcreteProduct** · **Creator** (declares `factoryMethod()`) · **ConcreteCreator** (overrides it, returns a ConcreteProduct) · Client.
Code smell	`if(type.equals("A")) new A(); else if ...` scattered in client code ⇒ tight coupling; every new type edits the client.

UML — GoF structure (Creator subclasses decide)



Dashed + hollow triangle = **implements** (realization). Solid + hollow triangle = **extends**. Dashed open arrow = **creates / depends on**. Creator is *abstract* (italic); `factoryMethod()` is abstract.

UML — Database creation (Quiz 2) · simple / parameterized factory



Java — both forms

```
// PRODUCT
public interface Database {
    void connect();
    void create(Product p);
}
// CONCRETE PRODUCTS
public class MySQLDatabase implements Database {
    public void connect() { System.out.println("MySQL
connected"); }
    public void create(Product p) { System.out.println("MySQL
insert " + p); }
}
public class PostgreSQLDatabase implements Database { ... }
public class MongoDatabase implements Database { ... }

// SIMPLE (PARAMETERIZED) FACTORY - the lecture form
public class DatabaseFactory {
    public static Database getDatabase(String type) {
        switch (type.toUpperCase()) {
            case "MYSQL":    return new MySQLDatabase();
            case "POSTGRES": return new PostgreSQLDatabase();
            case "MONGO":    return new MongoDatabase();
            default: throw new IllegalArgumentException(type);
        }
    }
}
// CLIENT - knows only the interface
Database db = DatabaseFactory.getDatabase(config.get("db"));
db.connect();
db.create(product);
```

```
// GoF FACTORY METHOD - subclass decides
public abstract class PaymentCreator {
    public abstract Payment createPayment();   // factory method
    public void pay(double amt) {              // uses the
product
        Payment p = createPayment();
        p.validate(); p.process(amt);
    }
}
public class CardPaymentCreator extends PaymentCreator {
    public Payment createPayment() { return new
CreditCardPayment(); }
}
public class PayPalPaymentCreator extends PaymentCreator {
    public Payment createPayment() { return new PayPalPayment();
}
}
public interface Payment { void validate(); void process(double
a); }
public class CreditCardPayment implements Payment { ... }
public class PayPalPayment implements Payment { ... }

// client
PaymentCreator c = new PayPalPaymentCreator();
c.pay(1500.00);
```

	Simple Factory	Factory Method (GoF)
Structure	1 class, 1 static method + switch	abstract Creator + ConcreteCreators
Who decides	parameter value	which subclass is used
New type	add class + edit switch	add product + creator (no edits, OCP)

Advantages / disadvantages / challenge → mechanism

Advantages	Disadvantages	Challenge → mechanism
Loose coupling: client depends on the interface, not concrete classes	More classes (one creator per product in GoF form)	Class explosion → use a parameterized simple factory / registry map
Centralized creation logic in one place	Switch in simple factory must be edited for new types	Edit switch → register types in a `Map<String,Supplier>` or config/reflection
Easy to extend (Open/Closed): add new product class	Extra abstraction for very simple cases	Invalid type requested → validate input, throw/return default
Hides complex construction; easier testing (mock product)	Client can't pick construction details	Need one shared instance → combine with Singleton

Why is `new` in client code bad? It hard-codes the concrete class ⇒ client must change for every new type (violates OCP, DIP). **SOLID:** OCP (extend with new products), DIP (depend on `Database` abstraction), SRP (creation separated from use).

Questions & model answers

In this section: **Database objects** FACTORY · **Library book types** FACTORY · **Bank accounts** FACTORY · **Payment objects** FACTORY · **LMS modules / framework** FACTORY · **Theory** FACTORY

FACTORY **DATABASE OBJECTS** — What is the role of the Factory pattern in the database scenario? *Quiz 2 Q1*

It **takes object creation out of the client**: the client asks `DatabaseFactory.getDatabase("MYSQL")` and receives a `Database`; the factory decides and instantiates `MySQLDatabase` / `PostgreSQLDatabase` / `MongoDatabase`. Client code works only through the `Database` interface.

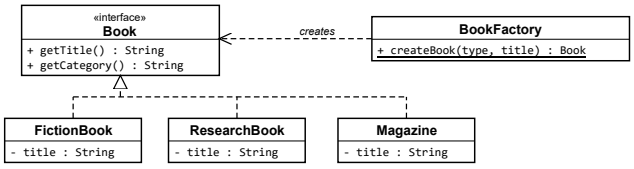
FACTORY **DATABASE OBJECTS** — State two advantages of the Factory pattern in this scenario. *Quiz 2 Q2*

(1) **Loose coupling** — switching DB (MySQL → PostgreSQL) needs no change in business code, only the parameter/config. (2) **Extensibility / centralized creation** — adding Oracle = one new class + one factory case; creation logic lives in one place, not repeated everywhere.

FACTORY **DATABASE OBJECTS** — Why doesn't the developer need to know whether the DB is MySQL or PostgreSQL? *Quiz 2 Q3*

Because of **abstraction + polymorphism**: the variable type is the `Database` interface and every concrete DB implements `connect()` / `create()`. The factory returns the correct object at runtime; the call `db.create(product)` runs the right implementation automatically.

FACTORY **LIBRARY BOOK TYPES** — Library: create Fiction, Research or Magazine books depending on user request. Book interface + BookFactory outline. *2025 Q3.2–3.3 (7)*




```
public interface Book { String getTitle(); String getCategory(); }
public class FictionBook implements Book {
  private String title;
  public FictionBook(String t) { title = t; }
  public String getTitle() { return title; }
  public String getCategory() { return "Fiction"; }
}  // ResearchBook, Magazine: same shape
public class BookFactory {
  public static Book createBook(String type, String title) {
    switch (type) {
      case "FICTION":  return new FictionBook(title);
      case "RESEARCH": return new ResearchBook(title);
      case "MAGAZINE": return new Magazine(title);
      default: throw new IllegalArgumentException("Unknown " + type);
    }
  }
}
Book b = BookFactory.createBook("RESEARCH", "AI Ethics");
```

FACTORY **BANK ACCOUNTS** — Banking system with Current, Saving and Children accounts. *2023 Q4.2 (5)*

Factory Method. `Account` interface/abstract class (`deposit`, `withdraw`, `calculateInterest`); `CurrentAccount`, `SavingAccount`, `ChildrenAccount` implement it with their own rules (overdraft, interest, withdrawal limit). `AccountFactory.createAccount(type)` returns the right one; the bank code works with `Account` only, so a new FixedDeposit type is added without touching it.

FACTORY **PAYMENT OBJECTS** — Payment: creating different payment objects (Credit Card, PayPal, COD). *2025 Q4.1(d)*

Factory Method — `PaymentFactory.create(method)` returns the right `PaymentProcessor` subclass; it combines with Template Method (each processor follows `handlePayment()`). Reason: client selects by user choice at runtime without `new`/if-else.

FACTORY **LMS MODULES / FRAMEWORK** — LMS needs different modules created depending on the user's needs. / Scalable, extensible product-based framework. *2022 Q4.5, Q4.2*

Factory Method. `Module` interface (Quiz, Forum, Assignment, VideoLecture); `ModuleFactory.create(userNeed)`. New modules/products = new classes registered with the factory, so the framework grows without changing existing code (Open/Closed) — exactly what a "product-based, extensible framework" needs.

FACTORY **THEORY** — "Code duplication can be avoided with this pattern." Agree?

Yes — creation logic (if-else + `new`) is written once in the factory instead of being repeated in every client; shared behaviour can sit in the abstract Creator. But concrete creators/products still add classes; keep the factory parameterized to avoid duplicating near-identical creators.

Comparisons & mistakes

vs Facade	Factory = **creates objects** ("which object?"). Facade = **coordinates existing objects** behind one interface ("what to do with many objects?").
vs Singleton	Factory controls *which type*; Singleton controls *how many*. Often combined (factory or created connection is a Singleton).
vs Abstract Factory	Abstract Factory creates *families* of related objects (not covered in detail).
Mistakes	Client still calls `new MySQLDatabase()` · factory returns the concrete type instead of the interface · forgetting the dashed "creates" arrow · calling it structural.